

Dokumentasi Teknis Pengembangan (Developer & Deployment Documentation)

SIP3K — Sistem Persuratan dan Penjadwalan Piket Kejaksaan

Dokumen ini ditujukan untuk **developer** yang akan melanjutkan pengembangan, melakukan maintenance, atau melakukan deployment proyek SIP3K. Untuk penjelasan proyek yang lebih umum/non-teknis, lihat [DOKUMENTASI - PROYEK.md](#).

Daftar Isi

- 1. Prasyarat & Setup Lingkungan Development
- 2. Arsitektur Aplikasi
- 3. Alur Autentikasi
- 4. Skema Database
- 5. Referensi API
- 6. Konvensi & Pola Kode
- 7. Cron Job & Tugas Terjadwal
- 8. Deployment ke Server (Docker)
- 9. Operasional Server
- 10. Troubleshooting
- 11. Panduan Kontribusi / Menambah Fitur

1. Prasyarat & Setup Lingkungan Development

Prasyarat

Tool	Versi Minimum	Catatan
Bun	1.x	Runtime & package manager utama proyek ini (bukan npm/yarn/pnpm).
Git	—	Untuk clone & version control.
Docker + Docker Compose	—	Hanya diperlukan untuk deployment / testing container secara lokal.

Node.js **tidak wajib** terpasang karena seluruh script (`dev`, `build`, `start`, `cron`, `db:seed`) dijalankan lewat Bun. Namun karena Next.js pada dasarnya kompatibel dengan Node, project juga bisa dijalankan dengan Node 20+ jika Bun tidak tersedia (ganti `bun` dengan `npm` / `node` pada perintah, dengan catatan lockfile `bun.lock` tidak akan terpakai).

Clone & Instalasi

```
git clone https://github.com/naufalhanif25/sip3k.git
cd sip3k
bun install
```

Menyiapkan Environment Variables

Salin `.env.example` menjadi `.env`, lalu isi nilainya:

```
cp .env.example .env
```

```
ADMIN_USERNAME=...      # username admin untuk login
ADMIN_PASSWORD=...      # HASH bcrypt dari password, bukan plain text
COOKIE_SECURE=...       # true / false – lihat catatan di bawah
JWT_SECRET=...          # string acak & rahasia, minimal 32 karakter disarankan
WA_API_TOKEN=...        # device token dari akun Fonnte
```

Soal `COOKIE_SECURE` : flag `secure` pada cookie `token` sekarang dikontrol manual lewat env var ini (`response.cookies.set("token", ..., { secure: process.env.COOKIE_SECURE === "true" })`), **bukan** otomatis mengikuti `NODE_ENV` lagi. Alasannya: cookie dengan `secure: true` hanya akan dikirim browser lewat koneksi HTTPS — kalau server production diakses lewat HTTP biasa (mis. lewat IP di jaringan lokal kantor tanpa sertifikat SSL), maka cookie tidak akan pernah terkirim balik dan **login akan selalu gagal** meski credentials benar. Jadi: - Set `COOKIE_SECURE=false` selama aplikasi diakses lewat HTTP (termasuk kondisi deployment saat ini di jaringan lokal Kejaksaan). - Set `COOKIE_SECURE=true` **hanya** setelah ada HTTPS aktif di depan aplikasi (misalnya Nginx dengan sertifikat SSL terpasang) — kalau di-set `true` tanpa HTTPS, gejalanya sama seperti bug: login sukses di response API tapi browser tetap balik ke halaman login.

Cara mengisi nilai `.env` / `.env.local` (lewat `data/rawenv.json` + `env:set`):

Alih-alih mengedit `.env` secara manual satu per satu (termasuk repot generate hash bcrypt sendiri untuk `ADMIN_PASSWORD`), cara yang dipakai proyek ini adalah lewat satu file sumber `data/rawenv.json` dan script `env:set` :

1. Buat/edit `data/rawenv.json` (file ini ada di dalam folder `data/` sehingga otomatis ter-`.gitignore`, **tidak pernah ikut ter-commit**), isi dengan nilai **mentah/plain text** untuk kelima variabel: `ADMIN_USERNAME`, `ADMIN_PASSWORD` (plain text — **belum** di-hash, `set-env.ts` yang akan meng-hash-nya otomatis), `COOKIE_SECURE`, `JWT_SECRET`, `WA_API_TOKEN`.
2. Jalankan:

```
bun run env:set:dev      # untuk lingkungan development lokal (src/scripts/set-env.ts)
# atau
bun run env:set:prod     # untuk server/di dalam container (dist/set-env.js)
```

3. Script ini akan otomatis: meng-hash `ADMIN_PASSWORD` dengan bcrypt, menuliskan hasilnya beserta variabel lain ke `.env` dan `.env.local`, serta menangani escaping karakter `$` pada hash bcrypt khusus untuk `.env.local` (`replaceAll("$", "\\$")`), diperlukan karena sebagian tool pembaca env menginterpretasikan `$` sebagai awal variable interpolation).

Cara ini lebih aman dan praktis dibanding generate hash manual satu-satu, terutama saat perlu mengganti password admin atau memperbarui kredensial di server — cukup ubah `data/rawenv.json`, jalankan ulang `env:set:dev` / `env:set:prod`, selesai. (Detail lebih lanjut soal `data/rawenv.json` dan `scripts/set-env.ts` ada di [bagian 7.4](#).)

Cara mendapatkan `WA_API_TOKEN` : login ke dashboard [Fonnte](#), scan/hubungkan device WhatsApp yang akan dipakai mengirim notifikasi, lalu salin device token dari dashboard tersebut — nilai ini yang dimasukkan ke `data/rawenv.json` sebelum menjalankan `env:set:dev` / `env:set:prod`.

Menyiapkan Data Awal

Proyek membutuhkan folder `data/` berisi `db.json` (database) di root proyek. Ada dua skenario:

- **Sudah punya `data/db.json` & `data/employees.json` dari server/lingkungan sebelumnya** → cukup taruh folder `data/` tersebut di root proyek. Folder ini sudah masuk `.gitignore` sehingga tidak akan ter-commit.
- **Instalasi baru dari nol** → siapkan `data/employees.json` (format lihat [bagian 4](#)) lalu jalankan:

```
bun run db:seed:dev
```

Perintah ini menjalankan `scripts/seed.ts`, yang akan **mereset** `data/db.json` menjadi `DEFAULT_DATA` yang `employees-`nya diisi dari `data/employees.json` (lihat `src/app/props/db.ts` → `DEFAULT_DATA`). **Jangan** jalankan `db:seed:dev` / `db:seed:prod` di server yang sudah punya data operasional (pickets/sptjb) karena akan menghapusnya — jika perlu memulihkan data yang tidak sengaja ter-reset, gunakan `bun run db:restore:dev` (lokal) atau `db:restore:prod` (server/Docker) — lihat [bagian 7](#) — untuk mengembalikan dari cadangan terakhir di `data/backups/`.

Menjalankan Mode Development

```
bun run dev
```

Berjalan di `http://localhost:3000` dengan Turbopack (hot reload cepat). Menyimpan data ke `data/db.json` relatif terhadap direktori kerja saat proses dijalankan (`process.cwd()`), jadi selalu jalankan perintah dari root folder proyek.

Catatan: cron job (`src/app/jobs/cron-picket.ts`) **tidak berjalan otomatis** saat `bun run dev`. Untuk menguji cron

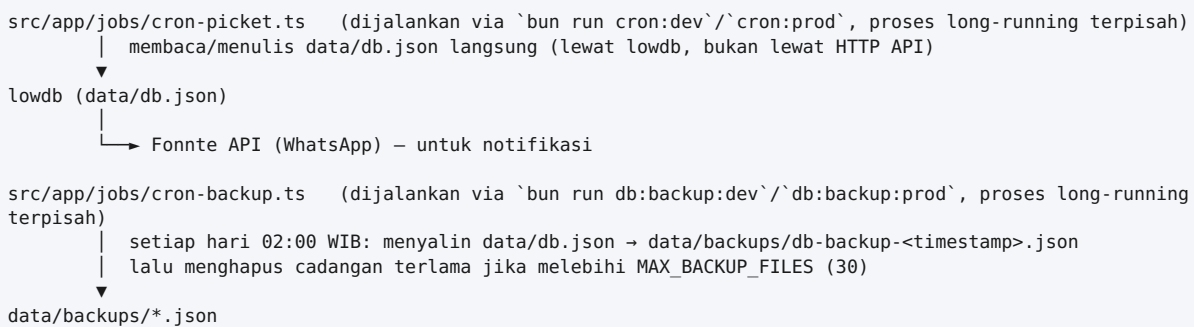
secara lokal, jalankan terpisah di terminal lain: `bun run cron:dev`.

2. Arsitektur Aplikasi

SIP3K adalah aplikasi **Next.js App Router** (bukan Pages Router) monolitik: frontend (React Server/Client Components) dan backend (Route Handlers di `src/app/api/`) berada dalam satu proyek yang sama, dijalankan sebagai satu proses Next.js.



Proses-proses **lain** yang berjalan terpisah dari server Next.js utama (masing-masing kontainer sendiri di Docker Compose):



Karena proses-proses ini (`sip3k-app`, `sip3k-cron`, `sip3k-backup` di Docker Compose) mengakses **file JSON yang sama** secara langsung (bukan lewat API/database server), penting untuk diperhatikan:

- `lowdb` tidak punya locking mechanism lintas-proses yang kuat — race condition (dua proses menulis `db.json` bersamaan persis di waktu yang sama) secara teori mungkin terjadi, meskipun risikonya kecil pada skala pemakaian aplikasi ini (operasi tulis jarang terjadi bersamaan: cron menulis 2x sehari, sedangkan aplikasi web menulis saat admin melakukan aksi CRUD).
- Kedua proses **harus** mengakses `data/db.json` yang sama persis (di Docker, ini dijamin lewat shared volume `./data:/app/data` pada `docker-compose.yml`).

Alur Routing Halaman (App Router)

Path Folder	URL	Tipe
<code>src/app/page.tsx</code>	<code>/</code>	Client Component — halaman login
<code>src/app/dashboard/page.tsx</code>	<code>/dashboard</code>	Client Component — redirect ke path pertama di <code>variables.paths</code>
<code>src/app/dashboard/sptjb/page.tsx</code>	<code>/dashboard/sptjb</code>	Client Component
<code>src/app/dashboard/picket/page.tsx</code>	<code>/dashboard/picket</code>	Client Component

Seluruh halaman menggunakan `"use client"` — aplikasi ini murni Client-Side Rendering untuk UI-nya, dan Route Handlers (`api/`) berfungsi sebagai backend yang dipanggil lewat `fetch` dari client (lihat pola di `src/app/lib/*-fetch-handler.ts`).

Middleware (`src/proxy.ts`)

Meski nama filenya `proxy.ts`, ini berfungsi sebagai **Next.js Middleware** (`export const config = { matcher: [...] }`) adalah signature middleware Next.js). Dijalankan di edge runtime sebelum request mencapai halaman:

```
matcher: ["/((?!api|_next/static|_next/image|favicon.ico|sitemap.xml|robots.txt).*)"]
```

Middleware ini **tidak** melindungi route `api/*` — proteksi endpoint API dilakukan secara terpisah di masing-masing `route.ts` lewat `verifyJWT()`.

3. Alur Autentikasi

1. **Login** (`POST /api/auth/login`): username/password dicocokkan dengan `process.env.ADMIN_USERNAME / ADMIN_PASSWORD` (single admin account — tidak ada tabel user multi-akun). Jika valid, JWT ditandatangani (`jsonwebtoken`, HS256 default) berisi payload `{ username }`, disimpan sebagai cookie `token` (`httpOnly`, `sameSite: lax`, `secure: process.env.COOKIE_SECURE === "true"` — lihat catatan `COOKIE_SECURE` di bagian 1).
2. **Setiap request ke API terproteksi**: membaca cookie `token`, memverifikasinya lewat `verifyJWT()` (`src/app/lib/global-utils.ts`, memakai library `jose`). Jika tidak valid/tidak ada → response `401 UNAUTHORIZED_RESPONSE`.
3. **Middleware (client-side page)**: mengecek token yang sama untuk redirect halaman (bukan proteksi data — data tetap diproteksi di level API).
4. **Logout** (`POST /api/auth/logout`): menghapus cookie `token` (`maxAge: 0`).

Catatan implementasi ganda JWT: signing memakai `jsonwebtoken` (di `api/auth/login`), verifikasi memakai `jose` (di `global-utils.ts`). Keduanya kompatibel karena sama-sama standar JWT dengan secret yang sama (`JWT_SECRET`), tapi kalau ingin konsistensi library saat refactor, keduanya bisa disatukan.

Masa berlaku token: **5 jam** (login biasa) atau **7 hari** (centang “Ingat saya”) — lihat `maxAgeInSeconds` di `api/auth/login/route.ts`.

4. Skema Database

Database adalah satu file `data/db.json`, divalidasi lewat skema **Zod** di `src/app/props/db.ts`. Struktur top-level:

```
{
  employees: Employee[],
  pickets: Picket[],
  sptjb: SPTJBDetail[]
}
```

4.1 Employee

```
{
  employeeId: string,      // NIP, wajib unik
  name: string,
  position: string,
  phone: string,           // format nomor Indonesia (divalidasi validator.isMobilePhone)
  gender: "L" | "P" | null,
  genderId: number,        // 0 = L, 1 = P (lihat variables.json → gendermap)
  rank: string,            // pangkat
  class: "I" | "II" | "III" | "IV" | null, // golongan
  classId: number,         // hasil deromanize dari `class`
  room: "a" | "b" | "c" | "d" | null,      // ruang
  roomId: number           // charCode(room) - 97, yaitu 0-3
}
```

4.2 Picket (satu batch = satu bulan)

```
{
  id: string,              // UUID
  startAt: Date,           // awal bulan
  endAt: Date,             // akhir bulan
  generatedAt: Date,       // waktu batch ini digenerate, default: waktu saat ini — dipakai untuk mengisi
  {month}/{year}/{date} pada kop surat ekspor jadwal (lihat data/template/picket.json)
  pickets: EmployeePicket[] // satu entri per hari dalam bulan tsb
}
```

`EmployeePicket` (satu hari piket):

```

{
  id: string,           // UUID
  employees: {
    first: EmployeePicketData, // dari class III
    second: EmployeePicketData // dari class II
  },
  supervisor: string,    // salah satu dari variables.supervisor, rotasi tiap 5 hari
  schedule: Date         // tanggal piket
}

```

`EmployeePicketData` adalah **salinan cache** dari sebagian data pegawai pada saat jadwal digenerate (employeeId, name, gender, phone, category) — bukan referensi/relasi ke `employees[]`. Konsekuensinya: kalau data pegawai (misal nomor HP) diubah **setelah** jadwal digenerate, jadwal piket yang sudah ada **tidak otomatis ikut ter-update**. Ini penting diketahui saat debugging laporan “nomor WA yang dikirim salah/lama”.

4.3 SPTJBDetail

```

{
  id: string,           // UUID
  docId: string,        // nomor surat, format: B-{0001}/{code}/SPTJB/{bulan}/{tahun}
  name: string,         // nama pembuat dokumen
  division: Division,   // salah satu dari variables.divisions
  _class: string,       // kode anggaran (budget class)
  createdAt: Date,
  updatedAt: Date,
  data: FormInput[]     // rincian belanja
}

```

`FormInput` (satu baris rincian belanja):

```

{
  code: number, name: string, description: string,
  date: Date, id: number,
  total: number, ppn: number, pph: number
}

```

4.4 File Pendukung: data/employees.json

Format flat (bukan Zod schema, langsung di-map ke `Employee` saat seeding — lihat `DEFAULT_DATA` di `db.ts`):

```
{ employeeid, name, position, phone, gender, genderid, rank, class, classid, room, roomid }[]
```

Perhatikan **penamaan key huruf kecil semua** (`employeeid`, bukan `employeeId`) — berbeda dengan skema `Employee` yang camelCase. Ini disengaja karena file ini adalah *raw seed data*, dipetakan manual ke skema resmi saat load.

5. Referensi API

Semua endpoint berbasis JSON (`Content-Type: application/json`). Endpoint bertanda **!** mewajibkan cookie `token` yang valid (mengembalikan `401` jika tidak).

Auth

Method	Endpoint	Body	Response
POST	<code>/api/auth/login</code>	<code>{ username, password, isRemember }</code>	<code>{ success, message, data: { username } }</code> + set-cookie <code>token</code>
POST !	<code>/api/auth/logout</code>	—	<code>{ success, message }</code> + hapus cookie <code>token</code>

Employee

Method	Endpoint	Body / Query	Response

GET 🚫	/api/employee	—	{ success, message, data: Employee[] } (terurut nama, A-Z)
POST 🚫	/api/employee	{ employeeId, name, position, phone, gender, rank, category } (category format "III/a" , dipecah jadi class + room)	{ success, message } — 400 jika field kurang/tidak valid, 400 jika employeeId sudah terdaftar
PUT 🚫	/api/employee	{ employeeId, position, phone, rank, category }	{ success, message } — 404 jika employeeId tidak ditemukan
DELETE 🚫	/api/employee?id={employeeId}	—	{ success, message } — 404 jika tidak ditemukan

Picket

Method	Endpoint	Body / Query	Response
GET 🚫	/api/employee/picket	—	Batch picket periode berjalan (atau batch terakhir jika tidak ada yang match tanggal hari ini)
POST 🚫	/api/employee/picket	—	Force-generate/regenerate jadwal picket bulan depan (menimpa batch yang sudah ada untuk periode tsb jika ada)
PUT 🚫	/api/employee/picket?id={picketBatchId}	{ id: scheduleId, employees: { first, second } }	Mengganti petugas picket (swap) pada satu hari tertentu dalam satu batch — 404 jika batch/jadwal/pegawai tidak ditemukan
GET 🚫	/api/employee/picket/today	—	Jadwal picket hari ini saja, data: null jika tidak ada
POST 🚫	/api/employee/notif	{ data: { name, phone }, date }	Kirim ulang notifikasi WA manual ke satu pegawai untuk tanggal tertentu

Dokumen SPTJB

Method	Endpoint	Body / Query	Response
GET 🚫	/api/document/sptjb	—	Semua dokumen, terurut nomor surat descending
GET 🚫	/api/document/sptjb?id={id}	—	Satu dokumen detail — 404 jika tidak ditemukan
POST 🚫	/api/document/sptjb	{ name, docId, division, data, _class }	Simpan dokumen baru, data.id = UUID baru
PUT 🚫	/api/document/sptjb?id={id}	{ name, division, data, _class }	Update dokumen — 404 jika tidak ditemukan
DELETE 🚫	/api/document/sptjb?id={id}	—	Hapus dokumen — 404 jika tidak ditemukan
GET 🚫	/api/document/template	—	Parsing public/template.xlsx → struktur cell/merge/header-footer terstruktur untuk dirender di frontend

Format respons standar (lihat `src/app/vars/db-vars.ts`):

```
// sukses
{ success: true, message: string, data?: unknown }
// gagal
{ success: false, message: string } // status 400/401/404/500 sesuai kasus
```

6. Konvensi & Pola Kode

Pola pemisahan lapisan (per fitur)

Proyek ini konsisten memakai pola berikut untuk tiap fitur (`employee` , `picket` , `sptjb` , `user`):

```
props/<fitur>.ts           → skema Zod + tipe TypeScript
lib/<fitur>-handler.ts      → logika perubahan state form (event handler React)
lib/<fitur>-fetch-handler.ts → pemanggilan API (fetch wrapper + parsing response pakai Zod)
lib/<fitur>*.ts             → logika bisnis murni (non-React), mis. algoritma penjadwalan
components/dashboard/<fitur>/ → komponen UI
hooks/<fitur>.ts           → custom hook React spesifik fitur
```

Saat menambah fitur baru, ikuti pola yang sama agar konsisten dengan kode yang ada.

Validasi data dengan Zod

Hampir semua data (input form, response API, data database) di-parse lewat skema Zod (`Schema.parse(data)`) sebelum dipakai. Ini berarti data yang tidak sesuai skema akan **throw error** saat parsing — pastikan endpoint API selalu dibungkus `try/catch` (sudah jadi konvensi di semua `route.ts` yang ada) supaya error tersebut tertangani jadi `500 SERVER_ERROR_RESPONSE` alih-alih meng-crash proses.

Path alias

`@/*` → `src/*` (diatur di `tsconfig.json`). Contoh: `@/app/lib/global-utils` = `src/app/lib/global-utils.ts` . Satu pengecualian: `@/./data/employees.json` (dipakai di `props/db.ts`) sengaja “naik” satu level dari `src/` untuk mengakses folder `data/` di root proyek — karena folder ini **di luar** `src/` .

Styling

Tailwind CSS v4 (config berbasis `postcss.config.mjs` + `@tailwindcss/postcss` , tanpa `tailwind.config.js` terpisah — sesuai konvensi Tailwind v4). Fungsi `cn()` (`clsx` + `tailwind-merge`) dipakai di hampir semua komponen untuk menggabungkan class secara aman (menghindari konflik class Tailwind yang saling override).

Linting & format

```
bun run lint
```

Konfigurasi ada di `eslint.config.mjs` (flat config, `eslint-config-next`). Ada juga `.prettierrc` untuk konsistensi format kode — disarankan pasang extension Prettier di editor dan aktifkan “format on save”.

Ringkasan File Konfigurasi Proyek

File	Fungsi
<code>next.config.ts</code>	Konfigurasi inti Next.js. Lihat penjelasan detail tiap opsi di bawah.
<code>Dockerfile</code>	Cetak biru pembangunan image Docker aplikasi (proses multi-stage <code>builder</code> → <code>runner</code>). Lihat penjelasan detail di bagian 8 .
<code>docker-compose.yml</code>	Konfigurasi untuk menjalankan keempat kontainer (<code>sip3k-app</code> , <code>sip3k-cron</code> , <code>sip3k-backup</code> , <code>sip3k-nginx</code>) sekaligus — termasuk pengaturan port, volume, environment variable, dan urutan/hubungan antar-service. Lihat penjelasan detail di bagian 8 .
<code>nginx/default.conf</code>	Konfigurasi reverse proxy Nginx (<code>sip3k-nginx</code>) yang menjadi pintu gerbang lalu lintas jaringan ke aplikasi. Lihat penjelasan detail di bagian 8 .
<code>eslint.config.mjs</code>	Konfigurasi ESLint (flat config, format baru ESLint 9+), extend dari <code>eslint-config-next</code> — aturan linting standar Next.js (deteksi hook React yang salah pakai, import yang tidak dipakai, aksesibilitas dasar, dll). Dijalankan lewat <code>bun run lint</code> .
<code>tsconfig.json</code>	Konfigurasi compiler TypeScript: target <code>ES2017</code> , mode <code>strict</code> aktif (validasi tipe ketat), path alias <code>@/*</code> → <code>src/*</code> , dan plugin <code>next</code> untuk dukungan tipe khusus Next.js (mis. tipe halaman/route).

<code>postcss.config.mjs</code>	Konfigurasi PostCSS, dipakai untuk memproses Tailwind CSS v4 (lewat plugin <code>@tailwindcss/postcss</code>) — menggantikan <code>tailwind.config.js</code> terpisah yang dipakai di Tailwind versi sebelumnya.
<code>.prettierrc</code>	Aturan format kode otomatis (indentasi, tanda kutip, dst.) untuk konsistensi gaya penulisan antar developer.
<code>.env.example</code>	Contoh format variabel rahasia yang dibutuhkan aplikasi (lihat bagian 1).
<code>.gitignore</code>	Daftar file/folder yang sengaja tidak ikut ter-commit ke Git (<code>node_modules</code> , <code>.next</code> , <code>data/</code> , <code>logs/</code> , file <code>.env*</code> kecuali <code>.env.example</code> , dst).
<code>.dockerignore</code>	Daftar file/folder yang tidak ikut disalin ke dalam image Docker saat build (<code>node_modules</code> , <code>.git</code> , <code>logs/*.log</code> , <code>data/backups/*</code> , dan beberapa file dokumentasi seperti <code>README.md</code> / <code>AGENTS.md</code> / <code>CLAUDE.md</code>) — mempercepat proses build & memperkecil ukuran build context.
<code>bun.lock</code>	File lock dependency milik Bun (setara <code>package-lock.json</code> / <code>yarn.lock</code>) — mengunci versi persis tiap dependency supaya instalasi konsisten di semua environment (<code>bun install --frozen-lockfile</code> dipakai di Dockerfile persis karena ini).
<code>AGENTS.md</code> / <code>CLAUDE.md</code>	Instruksi khusus untuk AI coding assistant (mis. Claude Code, Cursor, dll) yang dipakai tim saat mengembangkan proyek ini — bukan bagian dari aplikasi yang berjalan, murni catatan konteks untuk tools AI.

Penjelasan `next.config.ts`

```
const nextConfig: NextConfig = {
  reactCompiler: true,
  output: "standalone",
  experimental: {
    serverActions: {
      allowedOrigins: ["*"],
    },
  },
}
```

Opsi	Penjelasan
<code>reactCompiler: true</code>	Mengaktifkan React Compiler (fitur baru React yang otomatis mengoptimalkan re-render komponen, menggantikan kebutuhan manual <code>useMemo</code> / <code>useCallback</code> di banyak kasus). Bekerja sama dengan dependency <code>babel-plugin-react-compiler</code> di <code>devDependencies</code> .
<code>output: "standalone"</code>	Membuat hasil <code>next build</code> menghasilkan folder mandiri (<code>.next/standalone</code>) yang berisi server Node.js minimal + hanya dependency yang benar-benar dipakai — inilah yang memungkinkan image Docker produksi berukuran kecil , karena tidak perlu ikut menyalin seluruh <code>node_modules</code> .
<code>experimental.serverActions.allowedOrigins: ["*"]</code>	Perlu perhatian khusus — lihat penjelasan detail di bawah.

Soal `allowedOrigins: ["*"]` : Next.js secara bawaan punya proteksi keamanan untuk **Server Actions** (fitur Next.js untuk memanggil fungsi server langsung dari komponen client) yang disebut *origin checking* — secara default, Next.js hanya menerima permintaan Server Action yang asalnya (`Origin` header) **sama persis** dengan domain aplikasi itu sendiri, sebagai proteksi terhadap serangan CSRF (Cross-Site Request Forgery — situs lain mencoba memicu aksi di aplikasi ini atas nama pengguna yang sedang login, tanpa sepengetahuan mereka).

Dengan `allowedOrigins: ["*"]`, proteksi ini **dinonaktifkan** — Next.js akan menerima permintaan Server Action dari **origin/alamat manapun**. Ini kemungkinan besar sengaja diset begini karena aplikasi diakses lewat **berbagai alamat IP di jaringan internal** (bukan satu domain/alamat tetap), sehingga daftar origin yang perlu di-whitelist tidak bisa ditentukan di awal.

Implikasi yang perlu diketahui tim: - Selama aplikasi ini **hanya** mengandalkan pola fetch API biasa dari `lib/*.fetch-handler.ts` ke Route Handlers di `api/**/route.ts` (bukan fitur **Server Actions** Next.js — lihat [bagian 2](#), seluruh halaman

memakai "use client" + fetch()), maka pengaturan allowedOrigins ini **kemungkinan besar tidak berdampak** karena tidak ada Server Action yang benar-benar dipakai di kode saat ini — proteksi CSRF untuk endpoint API (route.ts) sendiri sudah ditangani terpisah lewat verifikasi JWT (lihat bagian 3), bukan lewat mekanisme ini. - **Namun**, jika di kemudian hari ada fitur baru yang memakai Server Actions ("use server"), pengaturan ["*"] ini berarti proteksi origin-checking bawaan Next.js untuk fitur tersebut **tidak aktif**. Selama aplikasi hanya bisa diakses dari jaringan internal Kejaksaan (tidak terbuka ke internet publik), risikonya relatif kecil — tapi ini sebaiknya ditinjau ulang (idealnya diisi daftar IP/domain spesifik, bukan "*") sebelum aplikasi dibuka ke akses publik atau sebelum ada fitur Server Actions baru yang menangani aksi sensitif.

7. Cron Job & Tugas Terjadwal

7.1 Pola Perintah :dev vs :prod

Sejak revisi terbaru, setiap perintah yang berkaitan dengan cron/database/env punya dua varian di package.json :

Kategori	Perintah :dev	Perintah :prod
Cron piket & notifikasi WA	cron:dev → bun run src/app/jobs/cron-picket.ts	cron:prod → bun run dist/cron.js
Seed database	db:seed:dev → bun run scripts/seed.ts	db:seed:prod → bun run dist/seed.js
Restore database	db:restore:dev → bun run scripts/restore.ts	db:restore:prod → bun run dist/restore.js
Backup database	db:backup:dev → bun run src/app/jobs/cron-backup.ts	db:backup:prod → bun run dist/backup.js
Set environment	env:set:dev → bun run scripts/set-env.ts	env:set:prod → bun run dist/set-env.js

Kenapa dipisah begini? Karena Bun bisa menjalankan file TypeScript (.ts) secara langsung tanpa perlu proses compile terpisah — cocok untuk iterasi cepat saat development (:dev , langsung baca src/ / scripts/). Tapi di Dockerfile , tahap builder mengompilasi kelima entry point ini jadi bundel JavaScript polos di dist/*.js (RUN bun build src/app/jobs/cron-picket.ts --outfile dist/cron.js ... , dst), dan tahap runner (image produksi final) **hanya menyalin folder dist/** — bukan src/ atau scripts/ mentah. Konsekuensinya: di dalam container produksi, **kode sumber .ts tidak ada sama sekali**, yang ada hanya dist/*.js hasil kompilasi. Karena itu, perintah :prod (yang menunjuk ke dist/) adalah **satu-satunya yang bisa jalan di dalam container**, sedangkan perintah :dev hanya bisa dipakai di lingkungan development lokal (yang punya src/ / scripts/ utuh).

Manfaat pemisahan ini: - **Keamanan**: kode sumber asli tidak ikut terbawa ke server produksi, memperkecil permukaan risiko kalau container diakses tidak sah. - **Ukuran & performa**: bundel dist/*.js sudah di-tree-shake/minify oleh bun build , sehingga lebih ringan dan start-up lebih cepat dibanding menjalankan TypeScript mentah lewat transpiler saat runtime. - **Kejelasan intent**: nama perintah langsung bilang lingkungan mana yang dituju, mengurangi risiko developer tidak sengaja menjalankan varian :dev di server produksi (yang sebenarnya juga tidak akan bisa, karena filenya tidak ada).

7.2 src/app/jobs/cron-picket.ts

Berisi 2 job berbasis croner , dengan timezone Asia/Jakarta :

Jadwal (cron expression)	Kondisi Tambahan	Aksi
0 8 * * * (setiap hari, 08:00)	Hanya dieksekusi jika hari ini = hari terakhir bulan berjalan	Generate jadwal piket bulan depan (skip jika batch untuk periode tsb sudah ada — lihat existingBatchIndex)
0 10 * * * (setiap hari, 10:00)	—	Cari jadwal piket besok , kirim WA ke 2 petugas via Fonnte. Isi pesan diambil dari template src/app/data/template/notif.md dan diisi otomatis lewat formatConfig() (mengganti placeholder {name} dan {date}) — untuk mengubah redaksi pesan notifikasi, cukup edit file notif.md tersebut, tidak perlu ubah kode.

```
bun run cron:dev # lokal, jalankan file .ts langsung
bun run cron:prod # server/Docker, jalankan dist/cron.js hasil compile
```

Di Docker, `sip3k-cron` menjalankan `cron:prod` (lewat `command: ["bun", "run", "dist/cron.js"]` di `docker-compose.yml`), terpisah dari `sip3k-app`.

7.3 `src/app/jobs/cron-backup.ts`

Satu job berbasis `croner`, timezone `Asia/Jakarta`:

Jadwal (cron expression)	Aksi
<code>0 2 * * *</code> (setiap hari, 02:00)	Menyalin <code>data/db.json</code> → <code>data/backups/db-backup-<YYYY-MM-DD_HH-mm-ss>.json</code> . Jika jumlah file cadangan di <code>data/backups/</code> melebihi <code>MAX_BACKUP_FILES</code> (30, lihat <code>vars/global-vars.ts</code>), file tertua dihapus otomatis (FIFO, berdasarkan urutan nama file yang sudah bertimestamp).

```
bun run db:backup:dev # lokal
bun run db:backup:prod # server/Docker (dipakai kontainer sip3k-backup)
```

Untuk restore manual dari cadangan terbaru:

```
bun run db:restore:dev # lokal
bun run db:restore:prod # server/Docker
```

(menjalankan `scripts/restore.ts` / `dist/restore.js`, **otomatis** mengambil file cadangan **terbaru** di `data/backups/` berdasarkan urutan nama file — bukan interaktif, tidak meminta memilih file secara manual.)

Proses-proses di atas (cron picket & cron backup) dijalankan sebagai **proses terpisah** dan berjalan terus-menerus (long-running, karena `croner` mendaftarkan jadwal lalu proses tetap hidup menunggu trigger berikutnya).

Debugging cron secara lokal tanpa menunggu jadwal asli: ubah sementara cron expression di file terkait (mis. jadi `* * * * *` untuk jalan tiap menit) saat testing dengan `cron:dev` / `db:backup:dev`, lalu **kembalikan ke expression semula sebelum commit**.

7.4 Folder `scripts/`

Dockerfile mereferensikan tiga file di `scripts/`, dikompilasi jadi `dist/seed.js`, `dist/set-env.js`, `dist/restore.js` (lihat 7.1 soal alasan dev/prod split-nya):

Script	Fungsi
<code>scripts/seed.ts</code>	Menjalankan <code>resetDatabase()</code> — mereset <code>data/db.json</code> ke <code>DEFAULT_DATA</code> (lihat bagian 1). Dijalankan lewat <code>db:seed:dev</code> / <code>db:seed:prod</code> .
<code>scripts/restore.ts</code>	Restore otomatis <code>data/db.json</code> dari file cadangan terbaru di <code>data/backups/</code> (urutan nama file, bukan interaktif). Dijalankan lewat <code>db:restore:dev</code> / <code>db:restore:prod</code> .
<code>scripts/set-env.ts</code>	Alat bantu generate/update <code>.env</code> dan <code>.env.local</code> dari satu file sumber <code>data/rawenv.json</code> (lihat catatan di bawah). Dijalankan lewat <code>env:set:dev</code> / <code>env:set:prod</code> .

Soal `data/rawenv.json`: file ini berisi nilai **mentah** (belum di-hash) untuk `ADMIN_USERNAME`, `ADMIN_PASSWORD` (plain text, akan di-hash otomatis oleh `set-env.ts` sebelum ditulis ke `.env`), `COOKIE_SECURE`, `JWT_SECRET`, `WA_API_TOKEN`. File ini ada di dalam folder `data/` sehingga otomatis ter-`.gitignore` bersama `db.json` / `employees.json` — **tidak pernah ikut ter-commit**. Ini adalah cara yang lebih praktis untuk mengatur ulang `.env` di server (cukup edit satu file sumber, lalu jalankan `bun run env:set:prod`) dibanding edit `.env` secara manual satu per satu, sekaligus otomatis menangani escaping karakter `$` pada hash bcrypt untuk `.env.local` (`replaceAll("$", "\\$")` — perlu karena beberapa tool baca env menginterpretasikan `$` sebagai awal variable interpolation).

Folder `scripts/` **sudah ikut disertakan di repo** (baris `/scripts` sudah dihapus dari `.gitignore`). Pastikan tetap ada di `main` sebelum build Docker, karena proses `docker compose up --build` akan gagal di step `RUN bun build scripts/*.ts` kalau folder ini hilang.

8. Deployment ke Server (Docker)

Ringkasan Arsitektur Container

```
docker-compose.yml
├─ sip3k-app      → bun run server.js (hasil `next build` mode standalone), expose 3000 (internal only)
├─ sip3k-cron     → bun run dist/cron.js (hasil compile cron-picket.ts)
├─ sip3k-backup   → bun run dist/backup.js (hasil compile cron-backup.ts)
└─ sip3k-nginx    → image nginx:alpine, publish 80:80, reverse proxy → sip3k-app:3000
```

`sip3k-app`, `sip3k-cron`, `sip3k-backup` dibangun dari `Dockerfile` yang sama (multi-stage: `builder` → `runner`), dijalankan dengan `command` berbeda di `docker-compose.yml`. `sip3k-nginx` memakai image resmi `nginx:alpine` tanpa build khusus, cukup mount config.

Perhatikan: `sip3k-app` sekarang pakai `expose` (bukan `ports`) — artinya port `3000` **hanya bisa diakses antar-container** dalam network Docker Compose yang sama, **tidak** terbuka ke host/publik. Satu-satunya port yang di-publish ke luar adalah `80` lewat `sip3k-nginx`.

Penjelasan `Dockerfile` (Multi-Stage Build)

```
FROM oven/bun:1-alpine AS builder
WORKDIR /app

COPY package.json bun.lock ./
RUN bun install --frozen-lockfile

COPY . .

ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1

RUN bun run build

RUN bun build src/app/jobs/cron-picket.ts --outfile dist/cron.js --target bun
RUN bun build src/app/jobs/cron-backup.ts --outfile dist/backup.js --target bun
RUN bun build scripts/seed.ts --outfile dist/seed.js --target bun
RUN bun build scripts/restore.ts --outfile dist/restore.js --target bun
RUN bun build scripts/set-env.ts --outfile dist/set-env.js --target bun

FROM oven/bun:1-alpine AS runner
WORKDIR /app

ENV NODE_ENV=production
ENV TZ=Asia/Jakarta
ENV NEXT_TELEMETRY_DISABLED=1

COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./next/static
COPY --from=builder /app/dist ./dist

EXPOSE 3000
CMD ["bun", "run", "server.js"]
```

Penjelasan baris per baris:

Bagian	Penjelasan
<code>FROM oven/bun:1-alpine AS builder</code>	Tahap pertama (diberi nama <code>builder</code>), dimulai dari image dasar Linux Alpine yang sudah terpasang Bun. Tahap ini akan “dibuang” di akhir — hanya hasil akhirnya yang dipakai.
<code>COPY package.json bun.lock ./</code> lalu <code>RUN bun install --frozen-lockfile</code>	Menyalin hanya file daftar dependency dulu (belum seluruh kode), lalu install. Trik ini disengaja: Docker akan meng-cache langkah ini, sehingga kalau kode berubah tapi dependency-nya tidak, proses <code>install</code> tidak perlu diulang dari nol setiap build — mempercepat build berikutnya.
<code>COPY . .</code>	Baru setelah dependency ter-install, salin seluruh kode sumber proyek ke dalam image tahap <code>builder</code> .
<code>RUN bun run build</code>	Menjalankan proses build Next.js (menghasilkan <code>.next/standalone</code> karena <code>output: "standalone"</code> di

	<code>next.config.ts</code> — lihat bagian 6).
5x <code>RUN bun build <file>.ts --outfile dist/<nama>.js --target bun</code>	Mengompilasi kelima entry point (<code>cron-picket.ts</code> , <code>cron-backup.ts</code> , <code>seed.ts</code> , <code>restore.ts</code> , <code>set-env.ts</code>) masing-masing jadi satu file JavaScript mandiri di <code>dist/</code> — inilah yang menghasilkan versi <code>:prod</code> dari script-script tersebut (lihat bagian 7.1).
<code>FROM oven/bun:1-alpine AS runner</code>	Tahap kedua (final), memulai image bersih baru — bukan melanjutkan tahap <code>builder</code> . Ini kunci dari multi-stage build.
4x <code>COPY --from=builder ...</code>	Hanya menyalin hasil jadi dari tahap <code>builder</code> : folder <code>public</code> , hasil build Next.js standalone, asset statis, dan folder <code>dist</code> . Kode sumber <code>.ts</code> mentah, <code>node_modules</code> penuh, cache build, dan tool build tidak ikut disalin — inilah alasan image akhir jauh lebih kecil, lebih cepat di-pull/dijalankan, dan lebih aman (kode sumber asli tidak ada di server produksi).
<code>EXPOSE 3000</code>	Menandai bahwa aplikasi di dalam container mendengarkan di port 3000 (dokumentasi/metadata untuk Docker, bukan yang membuka port ke luar — pembukaan port sesungguhnya diatur di <code>docker-compose.yml</code>).
<code>CMD ["bun", "run", "server.js"]</code>	Perintah default yang dijalankan saat container <code>sip3k-app</code> start — menjalankan server Next.js standalone hasil build. Kontainer lain (<code>sip3k-cron</code> , <code>sip3k-backup</code>) memakai image yang sama tapi meng-override perintah ini lewat <code>command:</code> di <code>docker-compose.yml</code> (mis. <code>bun run dist/cron.js</code>) — jadi satu <code>Dockerfile</code> yang sama dipakai ulang untuk tiga kontainer berbeda tugasnya.

Penjelasan `docker-compose.yml` (Per Service)

Service	Image / Build	Command	Port	Volume	Fungsi
<code>sip3k-app</code>	Build dari <code>Dockerfile</code> (tahap <code>runner</code>)	Default (<code>CMD</code> di <code>Dockerfile</code>) → <code>bun run server.js</code>	<code>expose: 3000</code> (internal only, tidak ke host)	<code>./public:/app/public</code> , <code>./data:/app/data</code> , <code>./logs:/app/logs</code>	Server web utama yang diakses pengguna (lewat Nginx).
<code>sip3k-cron</code>	Image sama dengan <code>sip3k-app</code>	Override: <code>bun run dist/cron.js</code>	—	<code>./data:/app/data</code> , <code>./logs:/app/logs</code>	Menjalankan <code>cron-picket.ts</code> (generate jadwal piket & kirim notifikasi WA terjadwal). <code>depends_on: sip3k-app</code> .
<code>sip3k-backup</code>	Image sama dengan <code>sip3k-app</code>	Override: <code>bun run dist/backup.js</code>	—	<code>./data:/app/data</code> , <code>./logs:/app/logs</code>	Menjalankan <code>cron-backup.ts</code> (backup database harian otomatis). <code>depends_on: sip3k-app</code> .
<code>sip3k-nginx</code>	<code>image: nginx:alpine</code> (tanpa build)	Default image	<code>80:80</code> (satu-satunya port yang publish ke host)	<code>./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro</code> (read-only)	Reverse proxy — pintu gerbang jaringan tunggal yang meneruskan semua traffic ke <code>sip3k-app:3000</code> lewat

					network internal Docker Compose. <code>depends_on:</code> <code>sip3k-app</code> .
--	--	--	--	--	---

Setiap service memakai `restart: always` (otomatis menyalakan ulang jika crash/server reboot) dan `environment: { NODE_ENV: production, TZ: Asia/Jakarta }` (kecuali `sip3k-nginx`, yang memakai image bawaan tanpa environment kustom). `env_file: [.env]` hanya diset untuk `sip3k-app` dan `sip3k-cron` — `sip3k-backup` **tidak memilikinya** (lihat catatan di bagian “Environment Variables di Docker” di bawah soal implikasinya). Ketiga service backend (`sip3k-cron`, `sip3k-backup`, `sip3k-nginx`) memakai `depends_on: [sip3k-app]`, yang memastikan urutan **start** container `sip3k-app` duluan — namun perlu diketahui `depends_on` di Docker Compose hanya menjamin urutan start container, **bukan** menunggu aplikasi di dalamnya benar-benar siap menerima koneksi (tidak ada health check di config saat ini).

Kenapa `sip3k-app`, `sip3k-cron`, `sip3k-backup` dipisah jadi 3 kontainer berbeda (padahal satu image yang sama)? Supaya masing-masing tugas berjalan **independen** — kalau `sip3k-cron` crash atau sedang di-restart, `sip3k-app` (yang diakses langsung oleh pengguna) tetap jalan tanpa gangguan, dan sebaliknya. Ini juga memudahkan pemantauan log/status per tugas secara terpisah (`docker compose logs -f sip3k-cron` vs `sip3k-app`).

Langkah Deployment (Instalasi Baru)

```
# 1. Clone repo di server
git clone https://github.com/naufalhanif25/sip3k.git
cd sip3k

# 2. Siapkan .env
cp .env.example .env
nano .env      # isi ADMIN_USERNAME, ADMIN_PASSWORD (hash bcrypt), COOKIE_SECURE, JWT_SECRET, WA_API_TOKEN

# 3. Siapkan folder data/ & nginx/
mkdir -p data logs
# taruh employees.json di data/ jika ada data pegawai yang sudah disiapkan sebelumnya,
# atau siapkan untuk db:seed (lihat bagian 1). Folder nginx/default.conf sudah ada di repo.

# 4. Build & jalankan (membangun & menjalankan ke-4 service sekaligus)
docker compose up -d --build

# 5. Jika instalasi baru (db.json belum ada), seed data awal
docker compose exec sip3k-app bun run db:seed:prod
```

Aplikasi bisa diakses lewat `http://<alamat-server>` (port 80, lewat Nginx) — **bukan** lagi `http://<alamat-server>:3000`.

Update ke Versi Baru

```
git pull origin main
docker compose up -d --build
```

Volume `./data` (termasuk `./data/backups`) dan `./logs` **tidak terpengaruh** oleh rebuild — data operasional aman. Jangan pakai `docker compose down -v` kecuali memang ingin menghapus volume.

Environment Variables di Docker

`docker-compose.yml` membaca `.env` lewat `env_file: [.env]` untuk service `sip3k-app` dan `sip3k-cron`, ditambah override eksplisit `NODE_ENV=production` dan `TZ=Asia/Jakarta` langsung di `environment:`. **Catatan:** service `sip3k-backup` pada `docker-compose.yml` saat ini **tidak** memiliki `env_file: [.env]` — hanya `NODE_ENV / TZ` manual. Karena `cron-backup.ts` tidak memerlukan variabel dari `.env` (tidak mengakses `ADMIN *` / `JWT_SECRET` / `WA_API_TOKEN`), ini bukan bug, tapi perlu diingat jika suatu saat job backup butuh salah satu variabel tersebut (mis. autentikasi ke penyimpanan cloud) — `env_file` perlu ditambahkan.

Reverse Proxy (Nginx — Sudah Terpasang By Default)

Berbeda dari versi sebelumnya, Nginx **bukan lagi opsional** — sudah jadi bagian baku arsitektur (`sip3k-nginx`), mengambil config dari `nginx/default.conf`:

```

server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://sip3k-app:3000;
        proxy_http_version 1.1;

        proxy_set_header Host $http_host;
        proxy_set_header X-Forwarded-Host $http_host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Port $server_port;

        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

```

Catatan penting soal config ini: - `proxy_pass http://sip3k-app:3000` memakai **nama service** Docker Compose (`sip3k-app`) sebagai hostname, bukan `127.0.0.1` — ini berfungsi karena Docker Compose otomatis membuat DNS internal antar-service dalam satu network yang sama. - Header `Upgrade / Connection: upgrade` disertakan untuk mendukung koneksi WebSocket/HMR (berguna kalau suatu saat ingin expose mode dev lewat proxy yang sama). - Config ini **hanya listen di port 80 (HTTP)**, belum ada blok `listen 443 ssl`. Untuk mengaktifkan HTTPS, tambahkan domain resmi + sertifikat SSL (mis. lewat Certbot/Let's Encrypt) ke `nginx/default.conf`, lalu ubah `COOKIE_SECURE=true` di `.env` setelah HTTPS aktif — lihat peringatan di [bagian 1](#) soal risiko login gagal kalau `COOKIE_SECURE=true` diaktifkan sebelum HTTPS benar-benar jalan.

9. Operasional Server

```

# Status seluruh container
docker compose ps

# Log real-time per service
docker compose logs -f sip3k-app
docker compose logs -f sip3k-cron
docker compose logs -f sip3k-backup
docker compose logs -f sip3k-nginx

# Masuk ke shell dalam container (debugging)
docker compose exec sip3k-app sh

# Restart satu service
docker compose restart sip3k-app

# Reload config Nginx tanpa restart penuh (mis. setelah ubah nginx/default.conf)
docker compose exec sip3k-nginx nginx -s reload

# Stop semua
docker compose down

# Backup manual database di luar jadwal otomatis (jalankan dari server host)
cp data/db.json data/backups/db-manual-$(date +%Y%m%d-%H%M%S).json

# Restore database dari cadangan
docker compose exec sip3k-app bun run db:restore:prod

```

Log aplikasi (dari `writeLog()` , bukan log Docker) tersimpan harian di `logs/YYYY-MM-DD.log` di dalam volume `./logs` , format:

```
[YYYY-MM-DD HH:mm:ss] [LEVEL] pesan
```

Berguna untuk audit trail terpisah dari `docker compose logs` (yang hanya menyimpan stdout/stderr sesuai konfigurasi logging Docker, dan bisa hilang jika container di-recreate tanpa log driver eksternal).

10. Troubleshooting

Gejala	Kemungkinan Penyebab	Solusi
Login gagal terus meski username/password benar	<code>ADMIN_PASSWORD</code> di <code>.env</code> belum di-hash (masih plain text) — biasanya karena <code>.env</code> diedit manual, bukan lewat <code>env:set</code>	Perbaiki lewat <code>data/rawenv.json</code> + <code>bun run env:set:dev / env:set:prod</code> , lihat bagian 1
Login berhasil di <code>curl</code> / Postman (dapat cookie <code>token</code>) tapi browser tetap redirect ke login	<code>COOKIE_SECURE=true</code> di <code>.env</code> padahal server masih diakses lewat HTTP biasa (belum ada HTTPS) — browser menolak mengirim balik cookie <code>secure</code> lewat koneksi HTTP	Set <code>COOKIE_SECURE=false</code> di <code>.env</code> selama belum ada HTTPS aktif, lalu <code>docker compose up -d</code> ulang agar env baru terbaca
Notifikasi WA tidak terkirim	<code>WA_API_TOKEN</code> salah/kadaluarsa, atau device Fonnte ter-disconnect	Cek dashboard Fonnte, scan ulang device jika perlu; cek <code>logs/</code> untuk pesan error dari <code>sendDirectWAMessage</code>
Jadwal piket bulan baru tidak ter-generate otomatis	Kontainer <code>sip3k-cron</code> tidak berjalan, atau job hanya trigger di hari terakhir bulan (by design)	Cek <code>docker compose ps</code> — pastikan <code>sip3k-cron</code> status <code>Up</code> ; generate manual lewat <code>POST /api/employee/picket</code> jika perlu segera
Data pegawai/piket/cadangan backup “hilang” setelah update/rebuild	Volume <code>./data</code> tidak ter-mount dengan benar, atau <code>docker compose down -v</code> terpakai (menghapus volume)	Pastikan <code>volumes: ./data:/app/data</code> ada di service terkait pada <code>docker-compose.yml</code> ; jangan pakai <code>-v</code> saat <code>docker compose down</code> kecuali memang ingin reset total
Error 500 tanpa pesan jelas di response	Data yang tersimpan tidak sesuai skema Zod (mis. hasil edit manual <code>db.json</code>)	Cek <code>logs/YYYY-MM-DD.log</code> untuk detail error dari <code>writeLog()</code> ; validasi <code>db.json</code> manual terhadap skema di <code>props/db.ts</code> ; jika perlu, restore dari cadangan (<code>bun run db:restore:prod</code>)
Nomor WA yang dikirim ternyata data lama pegawai	<code>EmployeePicketData</code> adalah cache saat jadwal digenerate, tidak sinkron otomatis dengan perubahan data pegawai setelahnya	Regenerate ulang batch piket terkait (<code>POST /api/employee/picket</code>), atau edit manual lewat fitur swap piket
<code>docker compose up --build</code> gagal di step build cron/seed/backup/set-env	Folder <code>scripts/</code> tidak lengkap/hilang dari direktori proyek	Pastikan <code>scripts/seed.ts</code> , <code>scripts/set-env.ts</code> , <code>scripts/restore.ts</code> ada sebelum build — lihat 7.4
Situs tidak bisa diakses sama sekali lewat <code>http://<alamat-server></code>	Kontainer <code>sip3k-nginx</code> tidak berjalan, atau <code>nginx/default.conf</code> tidak ditemukan/salah mount	Cek <code>docker compose ps</code> dan <code>docker compose logs sip3k-nginx</code> ; pastikan file <code>nginx/default.conf</code> ada di root proyek sebelum <code>docker compose up</code>

11. Panduan Kontribusi / Menambah Fitur

1. Buat branch baru dari `main` dengan prefix sesuai jenis kerjaan: `feat/`, `fix/`, `docs/`, `refactor/`.
2. Ikuti pola pemisahan lapisan yang sudah ada (lihat [bagian 6](#)) — jangan campur logika fetch API dengan logika UI dalam satu file komponen.
3. Tambahkan skema Zod di `props/` untuk setiap bentuk data baru, jangan pakai tipe `any` /tanpa validasi.
4. Jalankan `bun run lint` sebelum commit.
5. Jika mengubah skema data yang tersimpan di `db.json` (menambah/mengubah field), pertimbangkan efeknya terhadap **data lama yang sudah ada** — Zod `.default()` membantu memberi nilai fallback untuk field baru pada data lama, tapi field yang berubah tipe/dihapus perlu migrasi manual.
6. Push branch, buat Pull Request ke `main`, minta review sebelum merge.
7. Setelah merge, deployment ke server dilakukan manual lewat langkah di [bagian 8](#) (belum ada CI/CD otomatis).

Dokumen ini melengkapi `DOKUMENTASI-PROYEK.md` dan ditujukan untuk keperluan internal tim pengembang SIP3K.